



EAST WEST UNIVERSITY

Department of Computer Science and Engineering
B.Sc. in Computer Science and Engineering Program
Midterm Examination, Summer 2026 Semester

Course: CSE 207- Data Structures, Section-I
Instructor: Dr. Maheen Islam, Associate Professor, CSE Department
Full Marks: 30
Time: 1 Hour and 20 Minutes

$arr[0] = arr[1]$
 $arr[1] = arr[2]$
 $\vdots$
 $arr[i-1] = arr[i]$
 $arr[i] = arr[i+1]$
 $\vdots$
 $arr[n-1] = arr[n]$
 $arr[n] = 0$

Note: There are Five questions, answer ALL of them. Course Outcome (CO), Cognitive Level and Mark of each question are mentioned at the right margin.

1. Write a program that dynamically allocates memory for an array of integers. Find the largest value in the array and move it to the first position, keeping the relative order of all other elements unchanged. [CO1,C2, Mark: 5]

Example:

Input:

6 11 4 9 15 2

Output:

Largest element = 15

New Array = 15 6 11 4 9 2

temp = 15
arr[0] = temp
for i = 1 to n-1
arr[i] = arr[i-1]

temp = largest
ptr = temp = NULL

2. Write a program for a singly linked list that includes:
- A function to create the linked list
- A function to rearrange the linked list

[CO1,C3, Mark: 10]

The rearrangement must satisfy the following conditions:

1. Use the value of the first node as the pivot value.
2. All nodes having values smaller than the pivot must be moved before the pivot node.
3. All nodes having values greater than or equal to the pivot must remain after the pivot node.
4. The relative order of nodes in both groups must remain unchanged.
5. The rearrangement must be performed by modifying node links only. Do not copy data into new nodes.

Example:

Original List:

10 -> 4 -> 15 -> 7 -> 12 -> 3 -> NULL

Rearranged List:

4 -> 7 -> 3 -> 10 -> 15 -> 12 -> NULL

3. Given a doubly-linked list of integers, write a function to:
- Find the last positive value in the list
- Move that node to the beginning of the list
- Maintain the order of all remaining nodes

[CO1,C2, Mark: 5]

Example:

Input:

4 <-> 6 <-> -2 <-> 9 <-> 5

Output:

5 <-> 4 <-> 6 <-> -2 <-> 9

temp = next = newnode
if (temp != NULL)
temp->next = newnode
newnode->next = temp

4. Given a stack of integers, write a program to remove all occurrences of the minimum element from the stack.

[CO3,C2,
Mark: 5]

You are allowed to use one additional temporary stack, but recursion is not allowed. After removing the minimum element, the remaining elements must preserve their original relative order.

Example

Stack representation (bottom -> top):

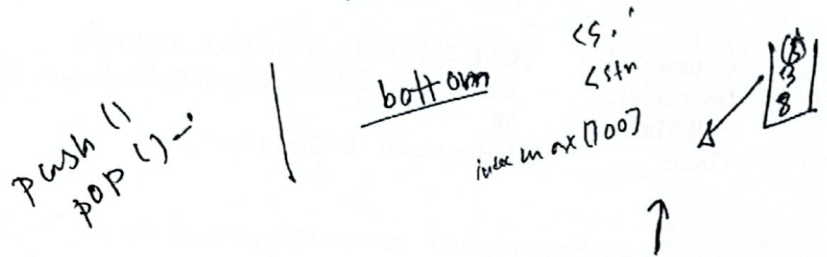
Input:

8 3 5 3 10 7 3

Minimum element = 3

Output:

8 5 10 7



5. Given a queue of integers, write a function to remove all even numbers from the queue. The relative order of the remaining odd numbers must remain unchanged. You may use one additional queue if necessary.

[CO3,C3,
Mark: 5]

Example:

Input (front -> rear):

4 7 10 3 8 5

Output (front -> rear):

7 3 5